We KANT, but we can
Group Leaders: Alessia Natalucci, Nico Montoya
Group Members: Katie Lai, Tiannnah Griffith
DS 4002
2 February 2026

Amazon Product Rating Analysis

Goal Statement
Be able to distinguish positive and negative amazon reviews for a product based on key words.

Research Question:
Can we run a model that will predict the rating of a product review with higher than 80% accuracy?

Modeling Approach
For our modeling approach we will use a Text CNN in order to classify Amazon reviews as positive and negative. We will use the text-based reviews of a product on Amazon and take key words or phrases such as “absolutely terrible” or “worth every penny” and will classify these into the two categories. Thus the model will be able to learn key phrases and be able to classify them. All of the text-based reviews will have a corresponding star-rating (out of 5) which we will be able to use to check the success of our model.

Executive Summary
This document outlines our data set establishment details and analysis plan for analysis of tone in Amazon Firestick product reviews.

Dataset Establishment Details
Our data is gathered from Amazon review research by UCSD. The data set was cleaned from the initial Electronics dataset presented by UCSD. This contained reviews on electronic devices from 2016-2023. This file contained multiple different products with billions of rows. We wanted to find the product with the most reviews and cleaned the rows to only contain the product with the most repeating ASIN (product identification number). We ended up with the Amazon Fire Stick which had over 100 thousand reviews.

katherinejlai Add files via upload 0ae0124 · 2 minutes ago History

Preview Code Blame 241 lines (241 loc) · 7.53 KB Raw

In [2]:
```python
import pandas as pd
from collections import Counter

asin_counter = Counter()

chunks = pd.read_json(
    "Electronics.jsonl",
    lines=True,
    chunksize=50_000
)

for chunk in chunks:
    asin_counter.update(chunk['asin'])

top_asin, top_count = asin_counter.most_common(1)[0]

top_asin, top_count
```

Out[2]: ('B00ZV9RDKK', 178239)

In [3]:
```python
rows = []

chunks = pd.read_json(
    "Electronics.jsonl",
    lines=True,
    chunksize=50_000
)

for chunk in chunks:
    rows.append(chunk[chunk['asin'] == top_asin])

df_top = pd.concat(rows, ignore_index=True)

df_top.head()
```

Out[3]:

	rating	title	text	images	asin	parent_asin	user_id	timestamp	helpful_vote	verified_purchase
0	4	Four Stars	Pretty cool!!! Thanks Amazon.	[]	B00ZV9RDKK	B075X8471B	AEM663T6XHZFWLODF4US2RCOCUSA	2017-10-31 23:07:42.120	0	True
1	5	Five Stars	We love it	[]	B00ZV9RDKK	B075X8471B	AEJDETWITK2KGACH7FUBMY33PPSQ	2018-03-01 21:29:17.127	0	True
2	5	AWESOME and simple to set up	i have hemmed and hawed for quite a while abo...	[]	B00ZV9RDKK	B075X8471B	AHGAOIZVODNHYMNCBV4DECZH42UQ	2018-07-25 00:03:16.921	0	True
3	4	Fire Stick Works great but can interfere with ...	Oct 2017 Review Update: Amazon has done a gre...	[]	B00ZV9RDKK	B075X8471B	AEDHXXX2F66EMWOSSPCQHS62UKNQ	2016-12-22 05:31:22.000	1	True
4	5	Would absolutely buy again for another tv	Love love love. Used to have Google Chromecast...	[]	B00ZV9RDKK	B075X8471B	AH2CJIKJPIIRDG264ZWX2UUQN5SQ	2018-03-28 13:38:10.750	0	True

In [5]:
```python
df_top.to_csv("most_reviewed_asin.csv", index=False)
```

Then we further cleaned the initial “most_reviewed_asin.csv” file. In this process, we dropped the columns images, parent_asin, helpful_vote, and user_id. The columns images and user_id were dropped as they were not necessary for the purposes of this project. The columns asin and parent_asin are both product ID’s, but we kept asin because "B00ZV9RDKK" is the product id for the amazon firestick, so we dropped any rows with a different ASIN because the row was a different product (around 5 rows were not dropped in the initial cleaning). We dropped parent_asin because it is similar to asin, but based on the model, color, and other aspects of the firestick, which were not needed for our project. The column helpful_vote tells us the number of people who found the review helpful, which was also unnecessary for our purposes since it doesn't indicate anything about the actual positivity or negativity of the review. Then, we also removed all rows that had "False" under verified_purchase and also reduced the timestamp column down to just the year.

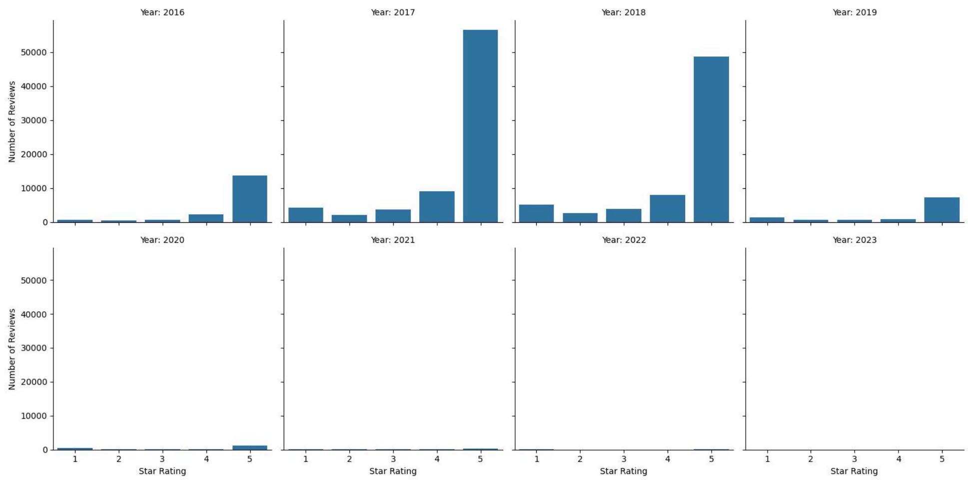
Data Dictionary:

Column	Description
rating	1 to 5 star rating of the product
title	Title of the review
text	Contents of the review
asin	Product ID code
timestamp	Year that the review was published
verified_purchase	Verification by Amazon that the reviewer purchased the product

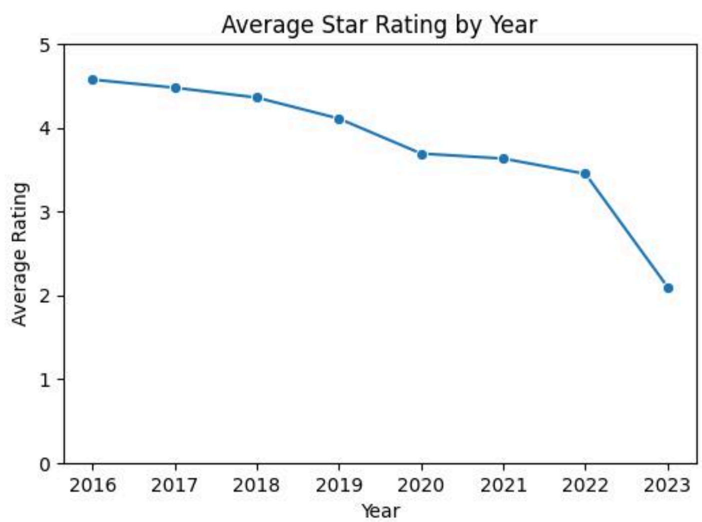
We explored the distribution of the dataset by year. The dataset is right skewed with the majority of reviews being from 2016-2019 and a tail from 2020-2023.
We also explored the distribution of the dataset by rating. The dataset is left skewed with the majority of reviews being 5-star.
We also found that the average rating per year is downward sloping from 2016-2023.

This brings us to the question of: is average rating downward sloping due to the distribution of the dataset? We can mitigate this through stratification while modeling if it is due to the distribution.

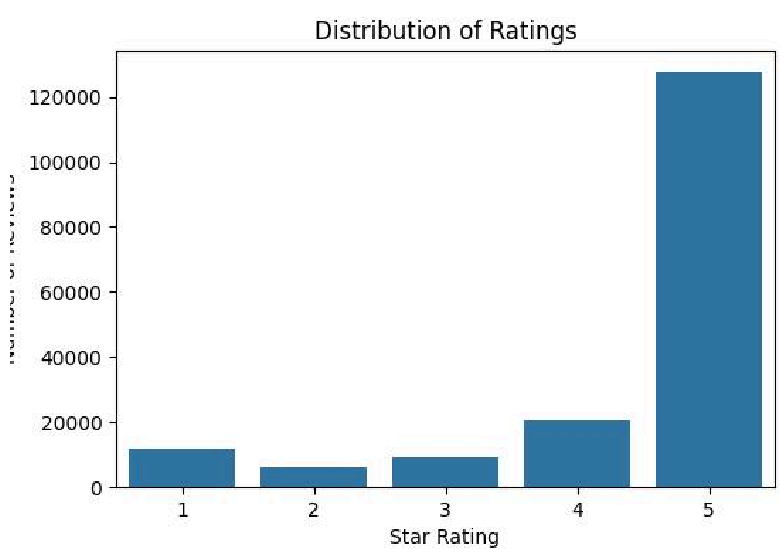
For our research question, we revised it to include a quantification.



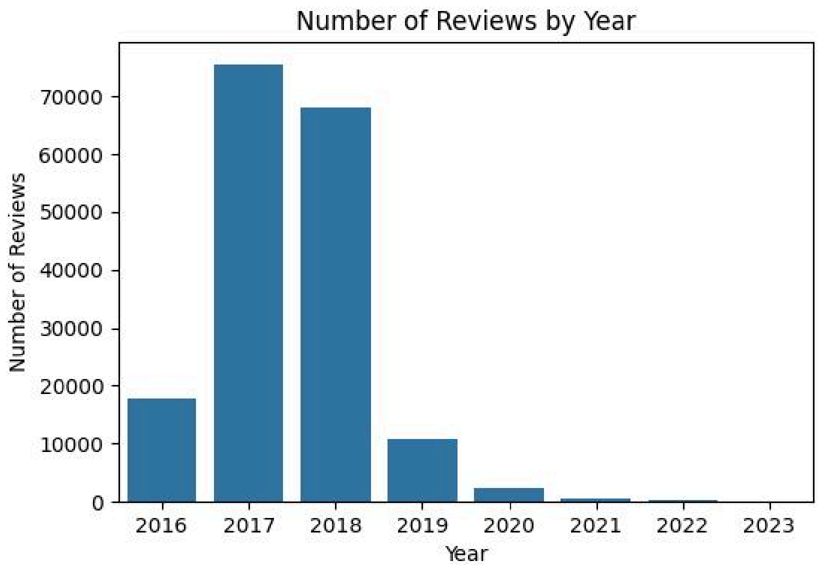
Year: 2016
Year: 2017
Year: 2018
Year: 2019
Year: 2020
Year: 2021
Year: 2022
Year: 2023
Number of Reviews
Star Rating

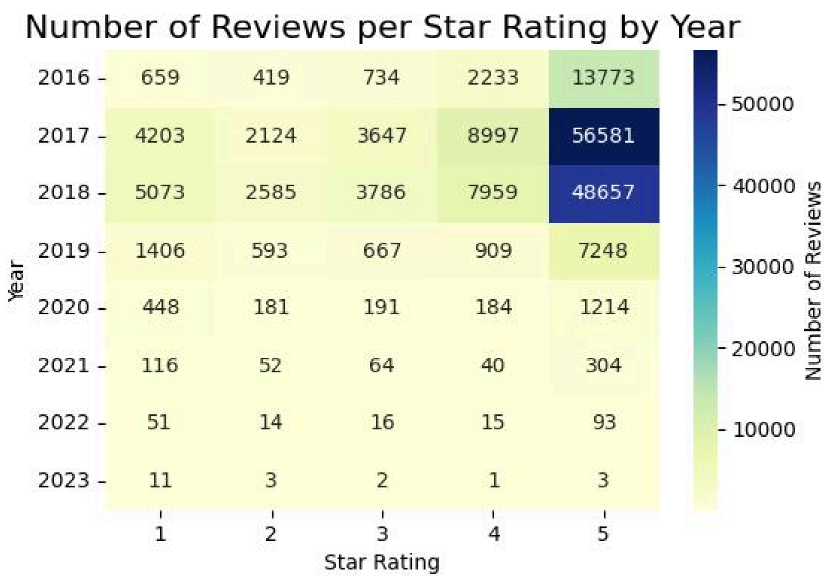


Average Star Rating by Year
Average Rating
Year



Distribution of Ratings
Star Rating





Analysis Plan

Step 1: Find Data and Preprocessing

- Search through approved data source locations to find a data set with respect to reviews for specific amazon products. After finding such a source, pick a product that has an ample amount of reviews to analyze. Once final selection is made, continue preprocessing. We cleaned the data as discussed in the Dataset Establishment Details section.

Step 2: Data Analysis

- We conducted EDA to understand the structure and characteristics of the dataset. This included examining the distribution of star ratings, the balance between positive and negative sentiment classes and the length of reviews. We used summary statistics to determine appropriate sequence length thresholds for text modeling. These insights informed our decisions on tokenization, padding length, and the architecture of the model.

Step 4: Text Representation

- We dropped 3-star reviews as they are ambiguous and cannot easily be classified as positive or negative. We then used a binary labeling of 0 or 1 to classify reviews as positive (1) or negative (0) with 4-5 stars as positive and 1-2 stars as negative. We then combined the text from the title and text columns and tokenized the text by assigning integers to words which turned it into a sequence of integers. All sequences are then padded to be the same length.

Step 5: Model Development (Our CNN Architecture)

- Each input token is mapped to embedding vectors and are applied to three convolution layers each with different kernel sizes. Global max pooling is then used to reduce the size of each input which is then concatenated and passed through a dropout layer to reduce overfitting. Then a sigmoid-activated dense layer outputs the predicted class. The model is trained using the Adam optimizer and binary cross entropy loss.

Step 4: Training Strategy and Data Splitting

- To evaluate the impact of dataset distribution on model performance, we implemented two training strategies. First, the dataset is split into training and validation sets using a stratified split, preserving the original class distribution. Second, an identical CNN is trained using a non-stratified split, which does not enforce class balance. All other training parameters, including architecture, batch size, and number of epochs, are kept constant so that we can have a fair comparison when answering our question that we were left with after EDA.

Step 6: Model Evaluation

- We evaluated model performance on the validation set using standard classification metrics, including accuracy, precision, recall, and F1 score. We also used cross entropy loss to assess the quality and confidence of predicted probabilities. We generated a confusion matrix to visualize classification performance and identify false positives and false negatives. The primary performance goal is to achieve accuracy, precision, recall, and F1 score of 80% or higher, and a cross entropy loss of 0.30 or lower.

Step 7: Comparative Analysis

- We will compare the evaluation metrics from the stratified and non-stratified models to assess whether class distribution significantly affects performance. We will analyze differences in recall, F1 score, and cross-entropy loss to determine the importance of maintaining class balance during training.

References

[1] R. Nandagopal, A. Jayakumar, and G. Manokaran, “Impact of Online Reviews on Consumer Purchasing Decisions.”